

实验三：跨平台应用开发

实验项目基本信息

- 实验编号：d20301035103、d20301009203
- 学时分配：4学时
- 实验类型：验证型
- 每组人数：6人
- 对应课程目标：课程目标2

核心步骤列表

实验概览

1. 需求分析阶段：定义列表页数据结构、API接口规范与传感器调用需求
2. AI辅助编码阶段：使用TRAE生成网络请求与JSON解析代码，辅助生成传感器调用模板
3. 测试验证阶段：模拟API异常响应测试容错能力，验证传感器数据采集
4. 部署运维阶段：多框架性能对比测试，输出框架适用性分析报告

开发任务清单

任务阶段	主要内容	关键技术点
Flutter列表	Dart + Provider	HTTP请求、JSON解析、下拉刷新
React Native列表	JSX + Hooks	Axios、FlatList、下拉刷新
Uniapp列表	Vue + Pinia	uni.request、下拉刷新
MAUI列表	C# + MVVM	HttpClient、数据绑定

成功标准

- ☐ 各框架列表页功能完整
 - ☐ RESTful API数据请求正常
 - ☐ JSON数据解析正确
 - ☐ 下拉刷新功能正常
 - ☐ 框架对比报告完成
-

实验任务与案例

核心任务

开发业务数据列表页（RESTful API网络数据请求 + JSON解析 + 下拉刷新），组内成员分别使用Flutter、React

Native、Uniapp、MAUI等不同框架实现同一功能需求。扩展案例：集成设备传感器（GPS定位、加速度计）。

实战案例

开发"智慧校园"新闻列表模块，从RESTful API获取新闻数据，解析JSON并展示列表，实现下拉刷新功能，体验跨平台框架的开发差异。

第一课时：Flutter与React Native开发（2学时）

3.1 需求分析阶段

步骤1：定义数据模型

```
{
  "code": 200,
  "data": {
    "items": [
      {
        "id": 1,
        "title": "智慧校园更新通知",
        "content": "系统升级通知...",
        "author": "管理员",
        "time": "2024-01-01 10:00:00"
      }
    ]
  }
}
```

步骤2：确定API接口

- URL: <https://api.example.com/news>
- Method: GET
- Response: JSON

3.2 Flutter列表开发

步骤1：创建Flutter项目

```
flutter create news_list
cd news_list
```

步骤2：添加依赖

```
# pubspec.yaml
dependencies:
  flutter:
```

```
sdk: flutter
http: ^1.1.0
provider: ^6.1.0
```

步骤3: AI辅助生成代码

1. 使用TRAE描述需求: Flutter新闻列表页, 下拉刷新, Provider状态管理
2. 生成代码片段

步骤4: 实现新闻列表

```
// main.dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => NewsProvider(),
      child: MyApp(),
    ),
  );
}

class News {
  final int id;
  final String title;
  final String content;
  final String author;
  final String time;

  News({required this.id, required this.title, required this.content,
    required this.author, required this.time});

  factory News.fromJson(Map<String, dynamic> json) {
    return News(
      id: json['id'],
      title: json['title'],
      content: json['content'],
      author: json['author'],
      time: json['time'],
    );
  }
}

class NewsProvider extends ChangeNotifier {
  List<News> _newsList = [];
  List<News> get newsList => _newsList;

  Future<void> fetchNews() async {
    final response = await http.get(
      Uri.parse('https://api.example.com/news')
    );

    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      final items = data['data']['items'] as List;
    }
  }
}
```

```

        _newsList = items.map((item) => News.fromJson(item)).toList();
        notifyListeners();
    }
}
}

class NewsListPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('智慧校园新闻')),
      body: Consumer<NewsProvider>(
        builder: (context, provider, child) {
          return RefreshIndicator(
            onRefresh: () => provider.fetchNews(),
            child: ListView.builder(
              itemCount: provider.newsList.length,
              itemBuilder: (context, index) {
                final news = provider.newsList[index];
                return ListTile(
                  title: Text(news.title),
                  subtitle: Text(news.author),
                  trailing: Text(news.time),
                );
              },
            ),
          );
        },
      ),
    );
  }
}

```

3.3 React Native列表开发

步骤1: 创建React Native项目

```

npx react-native-init NewsList
cd NewsList

```

步骤2: 安装依赖

```

npm install axios @react-native-community/refresh-control

```

步骤3: AI辅助生成代码

1. 使用TRAE描述需求: React Native新闻列表, FlatList, 下拉刷新
2. 生成代码片段

步骤4: 实现新闻列表

```

// App.js
import React, { useState, useEffect } from 'react';
import { FlatList, RefreshControl, Text, View, StyleSheet } from 'react-native';
import axios from 'axios';

const App = () => {
  const [news, setNews] = useState([]);

```

```
const [refreshing, setRefreshing] = useState(false);

const fetchNews = async () => {
  try {
    const response = await axios.get('https://api.example.com/news');
    setNews(response.data.data.items);
  } catch (error) {
    console.error(error);
  }
};

useEffect(() => {
  fetchNews();
}, []);

const onRefresh = async () => {
  setRefreshing(true);
  await fetchNews();
  setRefreshing(false);
};

const renderItem = ({ item }) => (
  <View style={styles.item}>
    <Text style={styles.title}>{item.title}</Text>
    <Text>{item.author} - {item.time}</Text>
  </View>
);

return (
  <FlatList
    data={news}
    keyExtractor={item => item.id.toString()}
    renderItem={renderItem}
    refreshControl={
      <RefreshControl refreshing={refreshing} onRefresh={onRefresh} />
    }
  />
);
};

const styles = StyleSheet.create({
  item: {
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#eee',
  },
  title: {
    fontSize: 16,
    fontWeight: 'bold',
  },
});

export default App;
```

第二课时：Uniapp、MAUI与传感器集成（2学时）

3.4 Uniapp列表开发

步骤1：创建Uniapp项目

1. 打开HBuilderX
2. 新建uni-app项目

步骤2：AI辅助生成代码

1. 使用TRAE描述需求：Uniapp新闻列表，uni.request，下拉刷新
2. 生成代码片段

步骤3：实现新闻列表

```
// pages/index/index.vue
<template>
  <view class="content">
    <view class="news-item" v-for="item in newsList" :key="item.id">
      <text class="title">{{item.title}}</text>
      <text class="info">{{item.author}} - {{item.time}}</text>
    </view>
  </view>
</template>

<script>
export default {
  data() {
    return {
      newsList: []
    }
  },
  onLoad() {
    this.fetchNews();
  },
  onPullDownRefresh() {
    this.fetchNews().then(() => {
      uni.stopPullDownRefresh();
    });
  },
  methods: {
    async fetchNews() {
      const res = await uni.request({
        url: 'https://api.example.com/news'
      });
      if (res.data.code === 200) {
        this.newsList = res.data.data.items;
      }
    }
  }
}
</script>
```

3.5 MAUI列表开发

步骤1：创建MAUI项目

1. 打开Visual Studio
2. 新建.NET MAUI项目

步骤2：AI辅助生成代码

1. 使用TRAE描述需求：C# MAUI新闻列表，HttpClient，MVVM
2. 生成代码片段

步骤3：实现新闻列表

```
// ViewModels/MainViewModel.cs
public partial class MainViewModel : ObservableObject
{
    [ObservableProperty]
    private ObservableCollection<NewsItem> _newsList = new();

    public async Task FetchNewsAsync()
    {
        using var client = new HttpClient();
        var response = await client.GetAsync("https://api.example.com/news");
        var json = await response.Content.ReadAsStringAsync();
        var data = JsonSerializer.Deserialize<NewsResponse>(json);

        NewsList = new ObservableCollection<NewsItem>(data.Data.Items);
    }
}
```

3.6 传感器集成扩展

步骤1：GPS定位实现（Flutter示例）

```
import 'package:geolocator/geolocator.dart';

Future<void> getLocation() async {
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) return;

    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
        permission = await Geolocator.requestPermission();
    }

    Position position = await Geolocator.getCurrentPosition();
    print('位置: ${position.latitude}, ${position.longitude}');
}
```

步骤2：加速度计实现（Flutter示例）

```
import 'package:sensors_plus/sensors_plus.dart';

accelerometerEvents.listen((AccelerometerEvent event) {
    print('X: ${event.x}, Y: ${event.y}, Z: ${event.z}');
});
```

3.7 测试验证与对比报告

步骤1：编写测试用例

- 1. 正常数据请求测试
- 2. 网络异常处理测试
- 3. 空数据处理测试

步骤2：输出框架对比报告

对比项	Flutter	React Native	Uniapp	MAUI
开发语言	Dart	JavaScript	Vue	C#
性能	★★★★★	★★★★☆	★★★☆☆	★★★★☆
学习成本	中等	低	低	中等

总结与思考

实验总结

- 掌握4种跨平台框架的列表页开发
- 学会RESTful API数据请求与JSON解析
- 了解移动设备传感器调用方法
- 完成框架适用性对比分析

课后思考

- 1. 各跨平台框架的适用场景分析
- 2. 如何选择合适的跨平台框架
- 3. 传感器在移动应用中的创新应用